1 Decision Boundaries

1A)
(i) No. Because this is a 3-dimensional range and if all x are at 0, then it will fail the and function. Likewise the invalid point will sit at the origin. And because any points on the decision boundary are considered misclassified, the singular point of (0,0,0) will invalidate the decision boundary.

(ii) A non-zero offset is allowed and necessary. As long as the offset doesn't go past a certain threshold.
$\vec{\theta}$= [1,1,1] b = -0.5
1B)
(i) Can classify
Because the positive values sit at $\sqrt{2}$away from the origin and the negative sit at 1 unit away from the origin, the circle can differentiate between positive and negative values. If the outside of the circle is positive and has a radius between 1 & $\sqrt{2}$.
(ii) Can't classify
No classifier exists, because the data points are not linearly separable through the origin. The positive points sit on a distinct linear line through the origin so an offset is required to properly classify the points.
(iii) Can classify.
Outside the square with side length between 1 and 3 (non-inclusive). And [a,b]= [-0.5,0.5] to name one example.
(iv) Can classify.
Outside of a square with $\alpha = 45$ so that the square is slanted that its corners sit on the axes. Length of the sides are in between $\sqrt{2}$ and $2\sqrt{2}$.

2 Perceptron algorithm with offset

2A)

$$\vec{\theta} = \sum_{i=1}^{\alpha_i} (y^{(i)} \vec{x}^{(i)})$$

$$b = \sum_{i=1}^{\alpha_i} (y^{(i)})$$

2B)
Distance from the origin to the decision boundary is defined by b, the offset. This value b, is then normalized by dividing against the normal vector θ which returns the direct distance from the origin to the closest point onto the decision boundary. By dividing against ||theta|| also provides information of the sign/direction. This therefore gives the shortest signed distance.

2D)
Received a theta of [4,0]
b: -6
total misclassification of 18

```
Done!
============== Classifier ==============
Theta:  [4. 0.]
b:  -6.0


============== Alpha ===================
i        Number of Misclassifications
========================================
0                1.0
1                0.0
2                11.0
3                2.0
4                0.0
5                4.0
Total Number of Misclassifications:  18.0
```

2E)
Yes, order matters. If the correct cluster of points are first introduced into the algorithm, the algorithm will start with being more accurately aligned, and therefore lower the number of mistakes seen/made. Since the points are linearly separable, order also helps converge the algorithm towards the right position. If too much clustered data is given, the algorithm may not converge properly until new data is introduced, therefore spaced out and distinct data will help the algorithm converge better/quicker.

3 Linear Regression

3.1D)
I don't think I did iterations correctly

Algorithm	learning_rate	θ_0	θ_1	# iterations	Runtime (s)
GD	10^{-4}	0.00073647	0.00038877	1	4.618699999991982e-05
GD	10^{-3}	0.00736466	0.00388771	1	5.059599999990283e-05
GD	10^{-2}	0.07364663	0.03887707	1	5.950200000044575e-05
GD	10^{-1}	0.73646626	0.38877072	1	9.161200000007419e-05
SGD	10^{-4}	0.00073549	0.00038827	20	4.063600000003831e-05

SGD	10^{-3}	0.00726803	0.00383845	20	3.661900000007101e-05
SGD	10^{-2}	0.06461495	0.03424944	20	3.871300000134556e-05
SGD	10^{-1}	0.23069147	0.11962253	20	7.010200000001632e-05
Closed form	-	0.35820909	0.01915174	-	0.000200669999999997046

3.1E)

GD has only 1 iteration compared to SGD 20 iterations, because it completely iterates through the whole data batch before updating theta. Whereas SGD updates theta for every data point it checks. It seems that overall SGD runtime is faster than that of GD when given similar learning rates. ie learning rate of 10^{-2} for both has a runtime of SGD = 0.0387 ms and GD = 0.0595 ms. Meaning that SGD runtime is faster since ~0.04ms < ~0.06ms.

The fastest runtime for SGD is with the learning rate of 10^{-3}. And the fastest runtime for GD uses a learning rate of 10^{-4}.

The coefficients both tend to reach around some value below 1. But varies heavily based on learning rate. For example Learning rate of 10^{-4} tends to ~0.000735 and ~0.000388, whereas learning rate of 10^{-1} is closer to ~0.23 (or ~0.74 with GD) and ~0.12 (~0.39 for GD).

This means that convergence is more accurate the smaller the learning rate is.

(rank 1 = shortest time, rank 9 = longest)

Rank	Table Order Above	Time (readable)
4	4.618699999991982e-05	0.0462 ms
5	5.059599999990283e-05	0.0506 ms
6	5.9502000000044575e-05	0.0595 ms
8	9.161200000007419e-05	0.0916 ms
==========	==========	==========
3	4.063600000003831e-05	0.0406 ms
1	3.661900000007101e-05	0.0366 ms
2	3.8713000000134556e-05	0.0387 ms
7	7.010200000001632e-05	0.0701 ms
==========	==========	==========
9	0.00020006699999997046	0.2001 ms

3.1F)

The runtime of closed-form is significantly longer than SGD. SGD generally has a runtime under 0.1ms (averaging around 0.0465ms) and the runtime for closed form reaches ~0.2ms. That is 4 times the time it takes for SGD.

Closest coefficients comes from SGD with a learning rate of 10^{-1}.

3.1G)

Proposed learning rate $\eta_k = \frac{1}{50k + 100}$

Since the learning rate of 10^{-2} performs oddly well compared to that of its GD counterpart, and the learning rate of 10^{-3} is the fastest of all the algorithms. I assumed the best learning rate would start around 10^{-2} then go towards 10^{-3}. Additionally the 10^{-4} and the 10^{-1} aren't optimal for their variant reasons. The first, seems to slow down due to how small the learning rate is and the other overshoots, which can be seen in the theta data, being vastly different from the other learning rates. Therefore to reach an optimal learning rate I had the step size adapt by $1/50$ per step. It starts at 10^{-2} so it doesn't flag out when there's 0 iterations. The number was derived from the fact that there are 20 iterations in this test data and between 1/1000 and 1/100 which is 1/45 per iteration. I changed it to 1/50 because I wanted the learning rate to get smaller faster, so there's no issues of overshooting.

After running the code the runtime is 8.288599999994872e-05 seconds, which is longer than the SGD with the constant learning rate of 10^{-1}.

Coefficients: [0.02108539 0.01137017]

The coefficients are in between the values returned from constant learning rates of 10^{-3} & 10^{-2}. The performance isn't great, though I suspect the code gets slowed down from calculating the learning rate within the function.

3.2 Linear Regression - Overfitting & Regularization

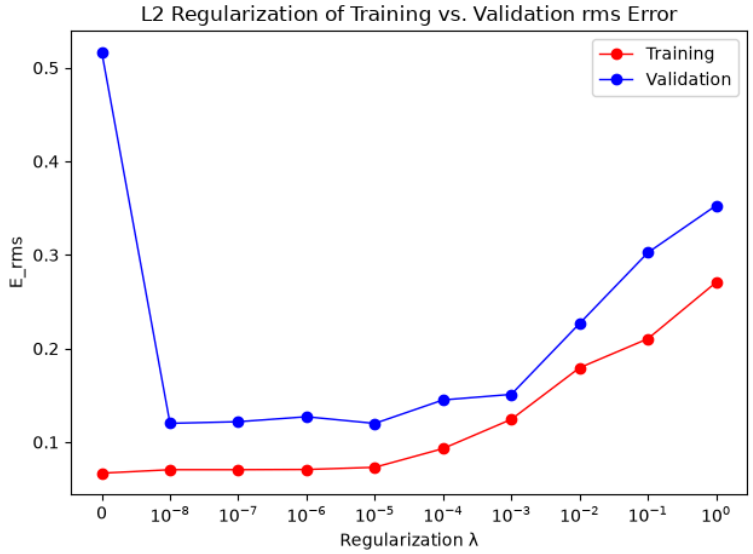
3.2B)



3.2C)

Degrees 4 through 8 best fits the data as the error is low for both training data & validation data. Underfitting occurs when there's high error for both data sets, since the model is not used to both sets, so around degrees 1 to 3 are underfitted. Overfitting occurs when the validation data has high error in comparison to the training data, because the model is built for the training data, but doesn't perform well on the actual validating data. Overfitted data will probably be on degrees 9 & 10.

3.2E)



3.2F)

$\lambda = 10^{-8}$ seems to be working the best. With the general range from around 10^{-8} to 10^{-5} working optimally. This is because both the training data and the validated data seem to have the lowest error compared to the other chosen lambda values.

3.2G)

I would probably model the data after the range of M (4,8) and λ (10^{-8}, 10^{-5}) because it seems to run well for both tests. However I would probably run various degrees (M) and lambda to get a better understanding of the relationship of the two values before modeling it after that. To be safe probably just model after degree 10 with a lambda somewhere along 10^{-8} to 10^{-5}. Since it already seems to perform well.